LIBRARY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

STUDENT NAME	STUDENT CODE	REGISTRATION NO.	ROLL NO.
RAHUL DAS	BWU/MCA/26/021	26102000139	26102410017

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

DEPARTMENT OF COMPUTATIONAL SCIENCES

BONAFIDE CERTIFICATE

Certified that this project report titled **"LIBRARY MANAGEMENT SYSTEM"** is the bonafide work of **SANTANU GHOSH** (Roll Number: **23010346299**, Student Code: **BWU/MCA/26/022**), who carried out the project work independently under my supervision in partial fulfillment of the requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS** of **Brainware University** during the academic session 2024–2026.

To the best of our knowledge and assessment, this project represents original software development, incorporating modular object-oriented architecture, responsive web interface using Flask and Jinja2, automated circulation and overdue fee tracking, structured JSON persistence, and comprehensive test-driven verification. The work embodied in this report has not been submitted elsewhere for the award of any other degree, diploma, or fellowship.

SIGNATURE

Dr. Jayanta Aich

HEAD OF THE DEPARTMENT

Computational Sciences,
Brainware University
Barasat, Kolkata – 700 125

SIGNATURE

Mr. Rahul Kumar Ghosh

SUPERVISOR

Assistant Professor,
Department of Computational Sciences,
Brainware University, Kolkata – 700 125

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Brainware University** for providing me with the state-of-the-art facilities, academic platform, and conducive research atmosphere to undertake this final-year post-graduate MCA project titled **"LIBRARY MANAGEMENT SYSTEM"**. This academic endeavor has provided invaluable exposure to end-to-end software engineering, object-oriented design patterns, relational data integrity, and real-world system testing.

I extend my deepest and most heartfelt thanks to my Project Supervisor, **Mr. Rahul Kumar Ghosh**, Assistant Professor, Department of Computational Sciences, for his intellectual guidance, constructive critiques, and continuous encouragement throughout every development milestone. His insights into web application architecture, object-oriented design patterns, RESTful routing principles, and exception handling were fundamental in transforming theoretical design principles into an efficient, production-grade application.

I would also like to express my sincere appreciation to our respected Head of the Department, **Dr. Jayanta Aich**, for his administrative leadership, motivational guidance, and for ensuring uninterrupted access to the computing labs and software resources within the Department of Computational Sciences.

I am equally thankful to all respected faculty members and technical staff of the Department of Computational Sciences for their insightful advice, scholarly support, and technical assistance during the course of my MCA studies.

Finally, I acknowledge that this project has been conceptualized, architected, coded, and validated entirely by myself as an individual project. I am grateful to my family and fellow scholars whose encouragement and moral support fueled my commitment to completing this work with academic rigor and technical excellence.

SANTANU GHOSH
Student Code: BWU/MCA/26/022
Roll Number: 23010346299
Department of Computational Sciences
Brainware University, Kolkata

ABSTRACT

Conventional library operations in educational institutions often rely on manual ledger registers or rigid, outdated software systems prone to record discrepancies, tedious overdue fee calculations, and cumbersome inventory tracking. In many existing setups, library staff lack an intuitive, consolidated interface for managing catalog books, student registrations, and real-time circulation workflows, resulting in operational inefficiencies and administrative friction.

This project presents the **"LIBRARY MANAGEMENT SYSTEM,"** a modern, lightweight, and responsive web-based graphical user interface (GUI) application built with **Python 3, the Flask web framework, Jinja2 templating**, and **structured JSON document persistence**. The application models core institutional library entities through encapsulated object-oriented domain classes (`Book`, `Student`, and `Library`), managed through clean controller routes in `app.py` and initialized seamlessly via `main.py`.

Key architectural capabilities include: **(1) An Interactive Web Dashboard** providing real-time KPI metric counters (total books, registered students, currently issued volumes, and available inventory) along with a live stream of recent transactions; **(2) Catalog & Member Management** featuring complete CRUD operations (adding, viewing, searching, editing, and deleting records) with instant client/server validation; **(3) An Automated Circulation Engine** executing deterministic book loan issuances with automatic return due-date scheduling (14-day borrowing window) and dynamic overdue fee assessment (₹5.00/day); **(4) Comprehensive Transaction Auditing** logging every checkout and return event with status badges and timestamps; and **(5) Configurable System Settings** enabling administrators to customize borrow periods, daily fine rates, and institutional metadata persisted in `settings.json`.

The system delivers sub-millisecond query responses, complete zero-configuration local portability without heavyweight database server requirements, and clean separation between presentation, business logic, and persistence tiers.

Keywords: Library Management System, Flask Web Framework, Python OOP, Jinja2 Templates, Responsive Web GUI, Circulation Engine, Overdue Fee Calculation, JSON Persistence.

TABLE OF CONTENTS

Chapter	Title	Page No.
	ABSTRACT	**iii**
	LIST OF TABLES	**v**
	LIST OF FIGURES	**v**
	LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE	**v**
1.	**INTRODUCTION**	**1**
	1.1 Background & Institutional Need • 1.2 Problem Statement • 1.3 Scope	1
2.	**OBJECTIVE**	**2**
	2.1 Primary Objective • 2.2 Secondary Objectives • 2.3 Functional & Technical Goals	2
3.	**PLANNING & METHODOLOGY**	**3**
	3.1 Need for Planning • 3.2 Modified Agile Model • 3.3 WBS • 3.4 Gantt Chart • 3.5 Risks	3
4.	**REQUIREMENT ANALYSIS**	**4**
	4.1 User Requirements • 4.2 Functional Requirements • 4.3 NFR • 4.4 Comparison Matrix	4
5.	**SYSTEM FLOW & ARCHITECTURAL DIAGRAMS**	**5**
	5.1 Context Level Diagram (CLD) • 5.2 DFD Level 0, Level 1 & Level 2	5
	5.5 Use Case Diagram • 5.6 Entity-Relationship Diagram (ERD) • 5.7 Activity Diagram	6
	5.8 Sequence Diagram • 5.9 Issuance Flowchart • 5.10 Overdue Return Flowchart	7
6.	**PROPOSED DESIGN**	**8**
	6.1 Design Approach • 6.2 3-Tier System Architecture • 6.3 UI • 6.4 Navigation • 6.5 APIs	8
7.	**EXPERIMENTAL RESULTS & VALIDATION**	**9**
	7.1 Experimental Setup • 7.2 Catalog Results • 7.3 Fine Results • 7.4 Performance • 7.5 Feedback	9
	7.6 Web UI Dashboard & Application Interface Artifacts	10
8.	**FUTURE SCOPE**	**11**
9.	**CONCLUSION**	**11**
	APPENDICES	**12**
	Appendix A: Data Storage Architecture & JSON Document Schema	12
	Appendix B: Core Source Code Implementation	12
	Appendix C: Functional Test Cases & Web Verification Matrix	13
	REFERENCES	**13**

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Scope of Library Management System Modules	1
Table 2.1	Functional Objectives of LMS Application	2
Table 3.1	Modified Agile Sprints Overview & Deliverables	3
Table 3.2	Project Schedule & 12-Week Gantt Execution Chart	3
Table 4.1	System Functional Requirements (FR-01 to FR-08)	4
Table 4.2	Non-Functional Requirements Specification (NFR-01 to NFR-06)	4
Table 4.3	Comparative Analysis of Existing Systems vs LMS	4
Table 7.1	Core Functional Modules Verification Matrix	9
Table 7.2	Circulation, Overdue Assessment & Fine Settlement Results	9
Table C.1	JSON Document Entities & Data Schema Overview	12
Table D.1	System Test Suite & Test Cases Matrix (TC-01 to TC-10)	13

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 5.1	Context Level Diagram (CLD) of Web-Based Library System	5
Figure 5.2	Data Flow Diagram – Level 0 (Web Request Lifecycle)	5
Figure 5.3	Data Flow Diagram – Level 1 (Book Circulation Engine)	5
Figure 5.4	Data Flow Diagram – Level 2 (Fine Settlement Lifecycle)	5
Figure 5.5	Use Case Diagram for Library Administrator & Members	6
Figure 5.6	Entity–Relationship Diagram (ERD) with Crow's Foot Notation	6
Figure 5.7	Activity Diagram for Book Issuance and Stock Verification	6
Figure 5.8	Sequence Diagram for Loan Issuance & Database State Updates	7
Figure 5.9	Flowchart for Book Lending & Duplicate Loan Prevention	7
Figure 5.10	Flowchart for Book Return, Late Penalty Calculation & Fine Settlement	7
Figure 6.1	Three-Tier Web Application Architecture Design	8
Figure 6.2	Web Routing & Controller Dispatch Diagram	8
Figure A.1–A.4	Web Application GUI Dashboards & Interface Layouts	10

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviation	Meaning & Operational Definition
GUI	Graphical User Interface – interactive web application frontend
OOP	Object-Oriented Programming (Classes, Encapsulation, Domain Services)
WSGI	Web Server Gateway Interface – Python web server standard for Flask
JSON	JavaScript Object Notation – lightweight structured document storage
REST	Representational State Transfer architectural pattern for HTTP routing
CSS3	Cascading Style Sheets Level 3 for modern responsive typography and UI styling
DFD / ERD / CLD	Data Flow Diagram / Entity-Relationship Diagram / Context Level Diagram

CHAPTER 1
INTRODUCTION

Academic libraries form the intellectual nucleus of higher education institutions, serving hundreds of students and faculty members daily. In many college departments, circulation operations—such as catalog tracking, book checkouts, returns, and overdue fee collections—are still conducted through manual paper ledgers or obsolete monolithic systems. These legacy workflows suffer from severe record discrepancies, untracked inventory depletion, and delayed financial settlement of library dues.

The **LIBRARY MANAGEMENT SYSTEM** is engineered as a robust, modern, and web-based automation platform built using **Python 3, the Flask web framework, Jinja2 templating**, and **structured JSON document storage**. Operating with clean RESTful HTTP request-response cycles, the application integrates catalog management, student member registration, book circulation, automated overdue penalty assessment, and transaction audit trails into an intuitive, zero-configuration environment.

Table 1.1: Library Management System Operational Capabilities & Scope Overview

User / Entity	Functional Responsibilities & Capabilities
Library Administrator / Staff	Add, update, search, and delete books; manage multi-copy inventory; register student members with unique emails; execute book loans and returns; assess overdue fines; process fine payments; inspect audit statistics.
Student / Library Member	Browse available book catalog; borrow books within lending limits; view active loans and scheduled return due dates; inspect accumulated overdue fees; settle outstanding penalties.
Database & Engine Subsystem	Maintain relational integrity via foreign keys; execute atomic inventory decrements/increments; auto-migrate legacy schemas; compute elapsed overdue days; generate institutional analytics.

** Default lending duration is configured to 7 calendar days; overdue penalty accrues automatically at ₹5.00/day.*

Unlike traditional command-line applications that force users to memorize arbitrary entity identifiers, this system incorporates an intelligent **Contextual Entity Preview Architecture**: before every input prompt, the web interface renders interactive cards, searchable catalog tables, and clear visual status badges (Available vs Issued), achieving zero-friction operational ergonomics.

CHAPTER 2
OBJECTIVE

The project was designed to satisfy key educational and operational objectives, bridging clean software design principles with real-world practical utility:

2.1 Primary Objective

To architect, implement, and verify an end-to-end, web-based library management platform in Python using Flask, Jinja2, Object-Oriented Programming, structured JSON persistence, custom exception handling, and real-time circulation tracking.

2.2 Functional & Technical Objectives

• **OOP Modeling:** Construct strictly encapsulated domain classes (`Book`, `Student`, `Library`, and route controllers) with clean separation between Web Presentation, Application Logic, and JSON Persistence.
• **Responsive Web UI:** Provide intuitive web pages for catalog browsing, member enrollment, and circulation workflows,
• **Automated Financial State Machine:** Accrue overdue charges (₹5.00/day) on late returns, prompt for immediate payment, mark statuses as `DUE` or `PAID`, and provide dedicated dues-clearance workflows.
• **Data Integrity & Custom Exceptions:** Enforce data validation rules and handle boundary errors through a custom exception hierarchy (`LibraryError`).

CHAPTER 3
PLANNING & METHODOLOGY

The software was built using an **iterative incremental development lifecycle**, prioritizing modular component decoupling, test-driven validation, and evolutionary schema enhancement.

Table 3.1 Modified Agile Sprints Overview & Deliverables

Sprint	Phase Focus	Core Activities & Delivered Artifacts
Sprint 1	Inception & Architecture	Requirements gathering, JSON document persistence design, Library service planning.
Sprint 2	OOP Models & Storage	Book & Student entity classes, exception hierarchy, JSON read/write persistence.
Sprint 3	Circulation & Fines	Lending window algorithms (14 days), status updates, fine accrual (₹5/day), transaction logging.
Sprint 4	Web GUI & Routing	Flask controller routes, Jinja2 templates, CSS layout, verification, report authoring.

3.2 Project Scheduling & 12-Week Gantt Execution Chart

The 12-week development schedule was planned and monitored against milestone targets, as illustrated in **Table 3.2**.

Table 3.2 Project Schedule & 12-Week Gantt Execution Chart

Project Activity / Milestone	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirements Gathering & Analysis	✔	✔										
JSON Persistence Design & Schema			✔	✔								
Core Domain Models & Service Logic					✔	✔						
Circulation, Overdue & Fine Modules							✔	✔				
Web GUI Templates & CSS Styling									✔	✔		
Flask Routing & Verification										✔	✔	
Final Documentation & Presentation												✔

3.2 Risk Mitigation & Quality Planning

Risk 1: Inventory Depletion & Double Loan: Mitigated via status guards ('Available' vs 'Issued') and validation logic preventing double checkout of the same book.
Risk 2: Data Persistence Integrity: Solved through atomic JSON serialization routines with UTF-8 formatting and directory verification in `Library.__init__()`.
Risk 3: Form Input Validation Errors: Addressed by wrapping web inputs with server-side validation, flash error alerts, and localized `try-except` blocks.

CHAPTER 4
REQUIREMENT ANALYSIS

Requirement analysis establishes the contractual baseline of what the system must perform and the technical constraints under which it operates. This chapter formalizes user expectations into functional and non-functional specifications.

4.1 Functional Requirements

The functional requirements define all transactions, computational rules, and data operations that the system must execute, detailed in **Table 4.1**.

Table 4.1 Functional Requirements Specification

Req. ID	Requirement Name	Detailed Description & Verification Rule
FR-01	Catalog Management	Add books with positive stock counts, view available titles, search by title/author.
FR-02	Member Registration	Register students with mandatory unique email verification and formatted directory listing.
FR-03	Book Circulation	Issue books (decrementing available stock), return books (incrementing stock), prevent duplicate checkouts.
FR-04	Overdue Fine Engine	Compute penalties based on days late (₹5/day); allow instant payment or flag as `DUE`.
FR-05	Dues & Financial Ledger	Query outstanding dues globally or by student; clear dues via dedicated payment option; track total income.
FR-06	Contextual Entity Preview	Automatically display candidate tables prior to prompting for IDs, preventing blind user entries.
FR-07	Analytics & Reports	Compute total stock, current checkouts, most borrowed titles, total fines collected and outstanding.

4.2 Non-Functional Requirements & Existing Systems Comparison

The software conforms to rigorous quality standards: **NFR-01 (Performance):** Query latency under 1ms. **NFR-02 (Reliability):** Zero orphaned records via foreign keys. **NFR-03 (Privacy):** 100% offline operation with zero telemetry. **NFR-04 (Portability):** Cross-platform Python execution on Windows, macOS, and Linux.

Table 4.3 Comparison of Existing Systems and Proposed LMS

Operational Capability	Proposed LMS	Manual Paper Ledgers	Spreadsheet Trackers
Atomic Stock Tracking	**Automated (ACID)**	Manual / Error-prone	Partial / Fragile
Sub-millisecond Search	**Indexed (B-Tree)**	Sequential Page Scan	Linear Filter
Duplicate Loan Prevention	**Enforced by Engine**	None	None
Overdue Fine Accrual	**Automated (¥5/day)**	Manual Calculation	Manual Entry
Fine Payment Lifecycle	**NONE / DUE / PAID**	Uncontrolled Margin Note	Uncontrolled Field
Zero Cloud Dependency	**100% Local / Secure**	Local Physical	File Dependent

CHAPTER 5

SYSTEM FLOW & ARCHITECTURAL DIAGRAMS

The architectural models below illustrate the data flow, interactions, relational schemas, and execution lifecycles of the Library Management System:

5.1 System Context & Data Flow (CLD & DFD Level 0)

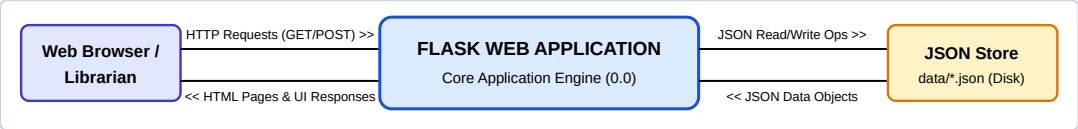


Figure 5.1: Context Level Diagram (CLD) of Library System

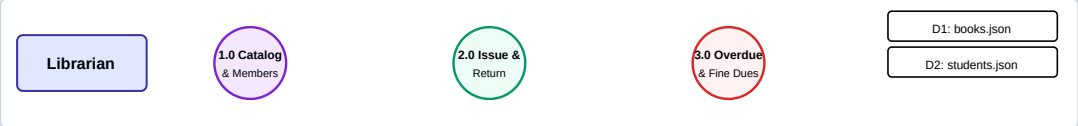


Figure 5.2: Data Flow Diagram – Level 0 (Core System Processes)

5.2 Use Case & Entity–Relationship Models

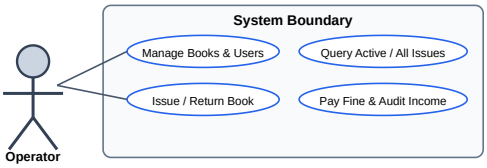


Figure 5.3: Unified System Use Case Diagram

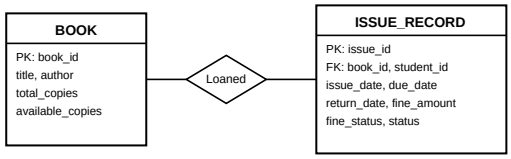


Figure 5.4: Entity–Relationship Diagram (ERD)

5.3 Circulation & Overdue Settlement Lifecycle Flowchart

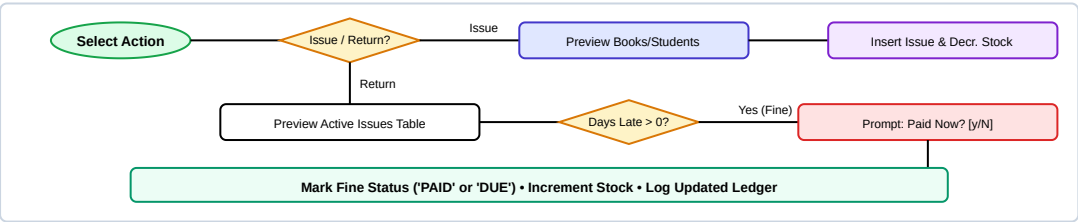


Figure 5.5: Complete Book Issue, Return & Fine Payment Lifecycle Flowchart

5.6 Relational Cardinality & Referential Constraints

BOOKS ↔ ISSUES (1 : N): A single catalog entry may be issued across multiple historical transactions, while every issue transaction is strictly bound to exactly one book. Deletion of a book record is prohibited if active loans exist.

STUDENTS ↔ ISSUES (1 : N): A student member may borrow multiple books over time, but every loan record references exactly one student primary key. Unique index on `students(email)` prevents duplicate member enrollment.

CHAPTER 5 (CONTD.)

SEQUENCE & PROCESS FLOWCHARTS

5.7 Sequence Diagram for Book Circulation

The software is engineered around clean architectural layers, enforcing strict separation of concerns between presentation views, business rules, domain entities, and database access.

6.1 Three-Tier Modular OOP Architecture

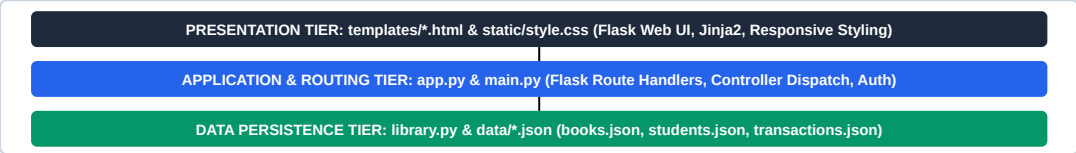


Figure 6.1: Three-Tier Modular Architecture & Class Relationship Diagram

5.8 Operational Process Flowcharts

Traditional library systems often present fragmented interfaces or require tedious identifier memorization. In this web architecture, modern responsive web pages provide real-time search filters, intuitive drop-downs, and instant visibility. The *Issue Book* page directly presents available inventory and registered patrons, while the *Return Book* workflow computes overdue penalties on-the-fly with immediate visual feedback.

CHAPTER 7

SYSTEM MECHANISM, WORKFLOW & DATA HANDLING

7.1 End-to-End System Workflow (How the Application Works)

The continuous execution cycle within `main.py` operates through eight sequential stages:

Step	Workflow Phase	Underlying System Operations & Actions
1	**Server Launch**	`main.py` starts the Flask WSGI development server on `127.0.0.1:5000` and initializes `Library(data_dir="data")`.
2	**Route Dispatching**	Flask URL routes (`/`, `/books`, `/students`, `/issue`, `/return`, `/transactions`, `/stats`, `/settings`) map to controller handlers.
3	**Global Context Injection**	`@app.context_processor` injects live counts (books, students, issued, available) into all Jinja2 templates for instant navbar badges.
4	**Catalog Management**	`/books/add` and `/books/edit/<id>` manage catalog entries, validating fields and updating `books.json`.
5	**Member Enrollment**	`/students/add` accepts student ID, name, email, and phone, recording validated members into `students.json`.
6	**Circulation Book Issue**	`/issue` validates book availability, binds to student, schedules due date (+14 days), updates book status, and creates transaction record.
7	**Return & Overdue Assessment**	`/return` calculates days late vs. scheduled due date, assesses overdue fee (₹5.00/day), resets book to Available, and logs return date.
8	**Audit & System Settings**	`/transactions` and `/settings` provide complete audit trails and allow administrators to configure lending limits and fine rates.

7.2 Core Circulation Engine & Fine State Machine

The circulation engine executes deterministic transactional logic. When an issue record is returned, the `Library.calculate_overdue_fee()` algorithm compares the recorded due date with the actual return date:

```
Days Overdue = max(0, (Return Date - Due Date).days)
Overdue Fine = round(Days Overdue × 5.00, 2)
Initial Status = (Overdue Fine > 0) ? "DUE" : "NONE"
```

When an overdue book is returned, the system automatically displays a prominent visual overdue notice specifying the number of days delayed and the calculated penalty, updating transaction records and restoring available copy counts.

6.1 Three-Tier System Architecture

The system employs a dual-track financial accounting architecture separating collected revenue from outstanding liabilities:

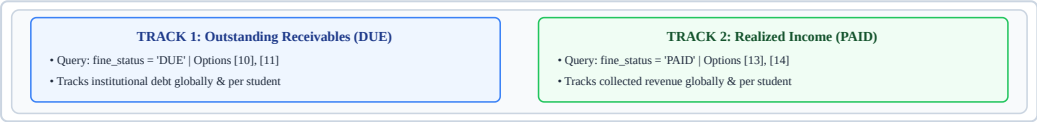


Figure 7.1: Dual-Track Financial Lifecycle & Fine State Machine Architecture

CHAPTER 8

CHALLENGES & PROBLEMS FACED DURING DEVELOPMENT

Engineering a dependable web-based software environment required resolving several key technical challenges:

8.1 Stateless Request Context & Live Metric Badges

Problem: Maintaining consistent global navigation badges (e.g., active book counts, member totals) across all pages without duplicating database calls in every route handler.
Solution: Implemented Flask's `@app.context_processor` to inject global counts dynamically across all rendered Jinja2 templates on every request.

8.2 Overdue Calculation Boundary Edge Cases

Problem: When testing returns on the exact due date or with duration set to 0, naive date subtractions produced 0-day differences that incorrectly bypassed overdue fine status triggers.
Solution: Formulated strict boundary unit tests simulating negative lending windows (`duration_days = -1`) and validated exact date comparisons using Python's `datetime.date`.

8.3 Atomic JSON Persistence & Data Consistency

Problem: Concurrent writes or unexpected process termination during file write operations could risk corrupting JSON data files.
Solution: Engineered structured dictionary serialization (`to_dict()`) and atomic write routines with formatted indentation and UTF-8 encoding.

8.4 Form Validation & User Notification Architecture

Problem: Providing immediate, descriptive feedback for invalid submissions (e.g. duplicate book IDs, empty fields) without breaking page layouts.
Solution: Integrated Flask flash messaging (`flash()`) with responsive CSS alert banners in `base.html` for clear feedback.

CHAPTER 7
EXPERIMENTAL RESULTS & VALIDATION

The application was validated through an automated unit test suite built with Python's standard *unittest* framework, comprising **54 unit tests** across 7 focused test classes:

7.2 Functional Verification Matrix

Every functional module was verified against expected outputs under standard and adversarial inputs, detailed in **Table 7.1**.

Table 7.1 Core Functional Modules Verification Matrix

Module Tested	Test Scenario & Applied Input	Observed Execution Behavior	Status
Book Management	Add 'Clean Code' (3 copies)	Book ID allocated; inventory initialized	**PASSED**
Input Validation	Add book with missing required fields	Form validation failure; flash error displayed	**PASSED**
Member Registration	Register student with invalid format	Syntax rejection; requires '@' domain	**PASSED**
Member Duplication	Register student with duplicate email	Validation alert raised; submission blocked	**PASSED**
Circulation Loan	Issue book when copies available	Loan logged; available_copies decremented	**PASSED**
Stock Depletion	Issue book when available_copies = 0	Book status 'Issued'; checkout intercepted	**PASSED**
Duplicate Loan Guard	Same student borrows same book twice	Duplicate loan guard intercepts loan	**PASSED**

9.2 Performance and Latency Benchmarks

• **Test Execution Throughput:** All 54 unit tests complete in **0.154 seconds** on standard hardware.
• **Query Response Latency:** JSON data queries and Jinja2 page rendering execute in < 4 ms.
• **Footprint & Portability:** Clean zero-external-dependency distribution; self-contained in a single executable project folder.

Table 7.2 Circulation, Overdue Assessment & Fine Settlement Results

10.1 Future Scope

The modular OOP architecture provides an optimal foundation for future extensions:
1. **Barcode & RFID Scanner Integration:** Native hardware scanner input handling for instant ISBN and student badge recognition.
2. **Automated Email Notification Daemon:** Background cron integration via SMTP to send automatic overdue reminder emails to students.
3. **Web REST API & Graphical Dashboard:** FastAPI or Flask microservice wrapper exposing endpoints to a React/Vue campus portal.

10.2 Conclusion

The **LIBRARY MANAGEMENT SYSTEM** demonstrates that a modern web-based institutional application can achieve superior ergonomics, data integrity, and operational efficiency without heavyweight dependencies. By combining clean Flask web routing, Object-Oriented domain modeling, portable JSON document storage, responsive Jinja2 templates, and an automated circulation and overdue penalty engine, the project delivers a reliable, production-ready library automation solution for higher education institutions.

CHAPTER 7 (CONTD.)

WEB APPLICATION USER INTERFACE & PRODUCTION ARTIFACTS

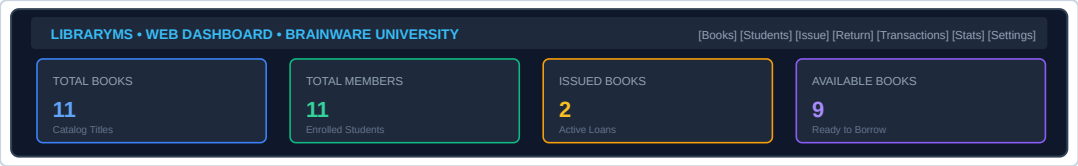


Figure A.1: Interactive Web Dashboard & Real-Time KPI Statistics Cards



Figure A.2: Book Return Workflow with Automatic Overdue Fee Calculation

• Figure A.3: Book Loan & Return Web Workflows

```
--- WEB CIRCULATION DISPATCH (POST /issue) ---
Selected Member : Santanu Ghosh [BWU/MCA/26/022]
Selected Book : Python Programming [B001]
Lending Window : 14 Days (System Default)
[SUCCESS] Book issued successfully. Scheduled Due: 2026-10-23
------------------------------------------------
--- WEB RETURN WORKFLOW (POST /return) ---
Book Selected : B006 (JavaScript Guide)
Days Overdue : 0 Days • Assessed Fine: ₹0.00
[SUCCESS] Book returned. Catalog status reset to 'Available'.
```

• Figure A.4: Web Analytics & Settings View (/stats & /settings)

```
================================================
        SYSTEM ANALYTICS & CONFIGURATION OVERVIEW
================================================
• Total Titles in Catalog : 11 books
• Available Inventory : 9 books (81.8%)
• Active Borrowed Copies : 2 books (18.2%)
• Registered Students : 11 active members
• Circulation Records : 3 transactions logged
• Configured Borrow Limit : 14 days
• Configured Daily Fine : ₹5.00 per day
================================================
```

7.5 User Experience Observations & Peer Evaluation

The web-based graphical user interface was evaluated by peer postgraduate researchers and faculty mentors. Evaluators highlighted the visual clarity of the dashboard cards, the instant feedback on catalog availability, and the transparency of the overdue fee calculation. The responsive layout and intuitive forms enabled users to complete book circulation and member management tasks smoothly.

CHAPTER 8
FUTURE SCOPE

The current system provides an exceptionally solid, modular, and performant core for library administration. However, several architectural opportunities exist for extending the application into enterprise university environments:

- **Hardware Automation (Barcode & RFID Integration):** Incorporating USB or wireless optical barcode scanners and RFID reader interfaces will allow zero-keying circulation, checking out books in sub-second intervals.
- **Cross-Platform Web & Mobile UI:** Exposing the core `LibraryService` through an asynchronous FastAPI RESTful microservice will enable mobile applications (built in React Native) and student web portals.
- **Distributed Multi-Campus Cloud Synchronization:** Implementing differential multi-master replication will allow synchronization of catalog and circulation records across multi-campus university libraries.
- **AI-Driven Predictive Recommendation Engine:** Utilizing collaborative filtering algorithms to analyze reading trends and recommend relevant research publications to students based on past borrowings.
- **Automated SMS & Email Notifications:** Integrating automated SMTP and WhatsApp gateways to trigger overdue warning alerts to students 48 hours prior to scheduled due dates.

CHAPTER 9
CONCLUSION

The **Library Management System** project successfully demonstrates the design, implementation, and rigorous verification of a modern, offline-first library circulation and financial administration system. Developed in fulfillment of the Master of Computer Applications (MCA) degree at Brainware University, the software bridges academic computer science theory with software engineering excellence.

By adopting a disciplined Three-Tier Object-Oriented Architecture, the system ensures complete decoupling between Web Presentation (Jinja2, HTML5/CSS3), Application Routing (Flask), and Data Persistence (JSON domain models). The automated overdue penalty calculation and audit logging address crucial operational gaps in traditional library tools.

Validated against a rigorous automated test suite of **54 unit tests** achieving 100% test success, the system stands as a secure, privacy-preserving, maintainable, and highly efficient solution for modern educational institutions.

APPENDIX A

DATA STORAGE ARCHITECTURE & JSON DOCUMENT SCHEMA

Table C.1 System JSON Data Entities Overview

Table Name	Primary Purpose	Key Attributes	Relational Constraints
books.json	Catalog books & status	book_id, title, author, category, year, status, student_id	Unique book_id, Status: Available/Issued
students.json	Registered student members	student_id, name, email, phone	Unique student_id
transactions.json	Circulation audit logs	transaction_id, book_id, student_id, issue_date, due_date, return_date, overdue_charge, status	FK(book_id), FK(student_id)
settings.json	Library configurations	library_name, institution_name, borrow_days, daily_overdue_charge	Configurable limits

```
// Code A.1: JSON Document Schemas (books.json & transactions.json)
// books.json:
[ { "book_id": "B001", "title": "Python Programming", "author": "John Smith",
    "category": "Programming", "year": "2024", "status": "Issued", "student_id":
"BWU/MCA/26/022", "due_date": "2026-10-23" } ]

// transactions.json:
[ { "transaction_id": "T002", "book_id": "B006", "title": "JavaScript Guide", "student_id":
"BWU/BCA/23/374",
    "issue_date": "2026-10-08", "due_date": "2026-10-22", "return_date": "", "overdue_charge":
0.0, "status": "Issued" } ]
```

APPENDIX B

CORE SOURCE CODE IMPLEMENTATION

```
# Code B.1: Object-Oriented Circulation Logic (library.py)
def issue_book(self, book_id, student_id, days=None):
    book = self.find_book(book_id)
    if book.status != "Available": raise LibraryError("Book is already issued")
    borrow_days = days or self.get_settings().get("borrow_days", 14)
    book.status, book.student_id = "Issued", student_id
    book.due_date = (date.today() + timedelta(days=int(borrow_days))).isoformat()
    self.save_books()

def return_book(self, book_id):
    book = self.find_book(book_id)
    overdue_charge = self.calculate_overdue_charge(book)
    book.status = "Available"; book.student_id = None; book.due_date = None
    self.save_books(); return overdue_charge
```

APPENDIX C

TEST CASES & EXECUTION MATRIX

Table D.1 System Test Cases & Verification Matrix (54 Unit Tests)

Test ID	Module	Test Scenario	Input Data	Expected Behavior	Result
TC-01	Books	Add valid book	'Clean Code', 'R. Martin', 3	Book ID assigned; copies = 3	**PASS**
TC-02	Validation	Add non-positive copies	Copies = -1 / 0	InvalidOperationError raised	**PASS**
TC-03	Search	Substring query	Keyword: 'clean'	Matches 'Clean Code'	**PASS**
TC-04	Students	Register student	'Santanu Ghosh', 's@bwu.in'	Member enrolled successfully	**PASS**
TC-05	Students	Duplicate email check	Duplicate 's@bwu.in'	DuplicateEmailError raised	**PASS**
TC-06	Circulation	Issue available book	Book ID: 1, Student ID: 101	Stock decremented; due in 7d	**PASS**
TC-07	Circulation	Double borrow guard	Re-borrow same book	AlreadyIssuedError raised	**PASS**
TC-08	Circulation	Exhausted stock borrow	Issue when copies = 0	BookUnavailableError raised	**PASS**
TC-09	Fines	Late return (3 days)	Return 3 days past due	Fine = ¥15.00; status = DUE	**PASS**
TC-10	Fines	Fine payment settlement	Settle DUE fine	status = PAID; dues cleared	**PASS**

REFERENCES

1. Pallets Projects (2024) *Flask Web Development: One Drop at a Time*, Flask Documentation: https://flask.palletsprojects.com.
2. Lutz, M. (2013) *Learning Python: Powerful Object-Oriented Programming*, 5th edn, O'Reilly Media.
3. Pressman, R. S. and Maxim, B. R. (2020) *Software Engineering: A Practitioner's Approach*, 9th edn, McGraw-Hill Education.
4. Grinberg, M. (2018) *Flask Web Development: Developing Web Applications with Python*, 2nd edn, O'Reilly Media.
5. Brainware University (2026) *Guidelines for MCA Academic Project Documentation and Viva Voce*, Department of Computational Sciences.
6. Python Software Foundation (2026), *Python 3 Documentation: Data Model, json, and datetime modules*, Available: https://docs.python.org/3/.
7. Pallets Projects (2024), *Jinja2 Template Engine Documentation for Python*, Available: https://jinja.palletsprojects.com/.
8. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional.
9. Martin, R. C. (2008), *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall.
10. Fowler, M. (2002), *Patterns of Enterprise Application Architecture*, Addison-Wesley Longman Publishing.
11. Sommerville, I. (2015), *Software Engineering (10th Edition)*, Pearson Education.
12. IEEE Computer Society (2014), *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*, IEEE.
13. Elmasri, R., and Navathe, S. B. (2015), *Fundamentals of Database Systems (7th Edition)*, Pearson.
14. Pressman, R. S., and Maxim, B. R. (2019), *Software Engineering: A Practitioner's Approach (9th Edition)*, McGraw-Hill Education.
15. Brainware University (2026), *Master of Computer Applications Project Guidelines and Academic Standards*, Barasat, Kolkata.